

Clase 128 — Integer overflows y errores aritméticos

Parte: 5 — Explotación de sistemas y binarios · Fuente: *The Shellcoder's Handbook* · CWE-190/191 🕒

Duración estimada: 100 min · Nivel: Intermedio

🎯 Objetivo

Comprender cómo los errores aritméticos con enteros (**overflow**, **underflow**, **truncamiento**, **confusión signed/unsigned**) se convierten en vulnerabilidades de memoria: una multiplicación de tamaño que se desborda genera un `malloc` demasiado pequeño y, después, un overflow de heap. Aprenderás a detectar estos patrones y a explotarlos indirectamente.

⚠️ **Ética:** solo en laboratorio propio.

📊 Resultados de aprendizaje

Al finalizar, el alumno podrá:

1. **Explicar** overflow, underflow, truncamiento y confusión de signo.
2. **Identificar** cálculos de tamaño vulnerables antes de `malloc / memcpy`.
3. **Provocar** un heap overflow a partir de un integer overflow.
4. **Detectar** estos bugs con UBSan y análisis de código.
5. **Programar** comprobaciones aritméticas seguras.

📖 Temas

#	Tema	Por qué importa
1	Representación de enteros y wrap-around	Base del comportamiento
2	Signed vs unsigned	Comparaciones que se invierten
3	Truncamiento (int→short)	Pérdida de bits altos
4	Overflow en cálculo de tamaño	<code>n*size</code> que se desborda
5	De integer bug a heap overflow	La cadena de explotación
6	Off-by-one	Un byte fuera de límite
7	UBSan	Detección en desarrollo
8	Aritmética segura	<code>__builtin_mul_overflow</code> , límites

Definiciones y características

- **Integer overflow:** el resultado excede el máximo del tipo y "da la vuelta". *Clave:* CWE-190; `0xFFFFFFFF + 1 = 0` en `uint32`.
- **Underflow:** una resta cae por debajo de 0 en unsigned → número enorme. *Clave:* `len - 1` con `len=0` da `SIZE_MAX`.
- **Truncamiento:** asignar un valor a un tipo más pequeño pierde bits altos. *Clave:* `int` → `short` puede pasar un tamaño grande a uno pequeño.
- **Confusión signed/unsigned:** una comparación firmada trata un negativo como válido. *Clave:* `if (len < MAX)` con `len` negativo pasa el chequeo y luego se usa como unsigned enorme.
- **Off-by-one:** escribir un elemento de más (típico `≤` en vez de `<`). *Clave:* puede sobrescribir el byte de metadatos del chunk siguiente.

Herramientas y preparación

```
gcc -fsanitize=undefined -g intov.c -o intov_ubsan
gcc -fsanitize=address -g intov.c -o intov_asan
```

Laboratorio guiado

Entorno propio.

1. Cálculo de tamaño vulnerable `intov.c`:

```
c #include <stdio.h> #include <stdlib.h> #include <string.h> char *dup_items(unsigned n){ //
BUG: n * 8 puede desbordar en 32 bits → buffer minúsculo unsigned size = n * 8; char *buf =
malloc(size); for (unsigned i = 0; i < n; i++) memcpy(buf + i*8, "AAAAAAA", 8); // heap
overflow return buf; } int main(int argc, char **argv){ dup_items(strtoul(argv[1],0,10)); }
```

2. Ejecuta con un `n` que provoque wrap-around (`n = 0x20000000` en 32 bits) y observa el crash.

3. Compila con UBSan y confirma el diagnóstico:

```
bash gcc -m32 -fsanitize=undefined -g intov.c -o intov32 ./intov32 536870912 # UBSan reporta
"unsigned integer overflow"
```

4. Con ASan, observa el `heap-buffer-overflow` que sigue al `malloc` insuficiente.

5. Corrige con aritmética segura:

```
c unsigned size; if (__builtin_mul_overflow(n, 8u, &size)) return NULL;
```

6. Ejercicio de confusión de signo: cambia el parámetro a `int` y pasa un valor negativo; observa cómo supera un chequeo `len < MAX` y luego explota como unsigned.

Ejercicios

1. Muestra un ejemplo de underflow (`len=0`; `len-1`) y su valor resultante.

2. Escribe un chequeo previo a `malloc` que rechace el overflow.
3. Provoca un truncamiento `int`→`short` y explica la pérdida de bits.
4. Detecta con UBSan un overflow signed y explica por qué es UB.
5. Encuentra un off-by-one en un bucle `for(i ≤ n)`.
6. Analiza un CVE real de integer overflow y resume la cadena hasta el heap overflow.

Reto verificable

Toma `intov.c`, demuestra el heap overflow con ASan y luego repáralo con `__builtin_mul_overflow`, probando que el mismo `n` malicioso ya no corrompe memoria.

Criterio de aceptación: antes del fix ASan reporta `heap-buffer-overflow`; después, el programa rechaza el tamaño y ASan no reporta nada.

Errores comunes

Síntoma / mensaje	Causa y cómo arreglar
No se desborda en 64 bits	El overflow depende del ancho del tipo; prueba en 32 bits o tipos pequeños
Chequeo <code>if (a+b < a)</code> optimizado	UB en signed; usa unsigned o <code>__builtin*_overflow</code>
UBSan no reporta	No compilaste con <code>-fsanitize=undefined</code>
Valor negativo "pasa" el límite	Comparación signed; convierte a unsigned con cuidado
memcpy tamaño enorme	Underflow en el cálculo; valida antes

Preguntas frecuentes

 **¿Un integer overflow es explotable por sí solo?** Normalmente no: su peligro está en alimentar después un `malloc` / `memcpy` / índice.

 **¿signed overflow es UB?** Sí en C: el compilador puede asumir que no ocurre, generando bugs sutiles. Unsigned hace wrap definido.

 **¿Cómo protejo mi código?** Valida tamaños con builtins de overflow, usa `size_t` correctamente y activa UBSan en CI.

Referencias

- CWE-190: Integer Overflow or Wraparound — <https://cwe.mitre.org/data/definitions/190.html>
- CWE-191: Integer Underflow — <https://cwe.mitre.org/data/definitions/191.html>
- UndefinedBehaviorSanitizer — <https://clang.llvm.org/docs/UndefinedBehaviorSanitizer.html>
- The Shellcoder's Handbook, cap. de integer bugs. Wiley.

Material descargable

-  [Guía en PDF](#) — versión imprimible de esta clase.

-  [Presentación \(PPTX\)](#) — deck para proyectar en clase.

Siguiente clase

[Clase 129 - Explotacion en Windows: manejo de SEH](#)